

Agile 방법론 및 MSA 개발

수업 운영 가이드

수업 진행 일정

- 1 일차 개념 실습은 오후 3 시부터 공통으로 사전 제공된 완성형 코드 템플릿을 스스로 이해하며 정리한 서브노트 초안을 작성합니다.
- 1 일차 종합 실습은 오후 4 시부터는 이에 대해 실제 본인이 스스로 적용하며 겪은 시행착오나 코드 템플릿을 이해하기 위한 과정 및 경험을 서브노트에 추가하여, 각 반별로 제출합니다. *각 반별 제출 슬랙 쓰레드는 담당 교수님이 생성
- 2 일차에는, 완성된 코드 템플릿을 이해하고 활용할 수 있다는 전제로, 팀별로 기획한 아이디어에 맞춰 커스터마이징하여 미니 AI 를 위한 MSA 서비스를 개발하는 방식입니다.
- 2 일차 실습은 다음과 같이 Agile 방법론에 따라서 2 개의 Sprint 로 진행
- → **Sprint1 워킹 스켈레톤 (walking skeleton)** 개념으로 시나리오 기획 : 시스템 전체 (**user, course, enrollment, kafka, payment, recommend**)를 동시에 조금씩 건드리는 게 아니라, 도메인 서비스를 먼저 골라서 (3 개) 그 안에서 사용자가 처음부터 끝까지 실제로 쓸 수 있는 한 가지 흐름을 완전하게 만드는 것)
- → **Sprint2 (확장) 2 단계로 진행** : 이어지는 다음 Sprint2 에서는 Sprint1 에서 손대지 않은 나머지를 다룹니다.
 - **Kafka 이벤트** : payment.completed 같은 이벤트 흐름을 붙여서, 결제 (또는 팀이 정의한 트리거) 완료 시 enrollment 상태가 자동으로 바뀌게 만드는 것 (토픽 이름·Producer/Consumer 구조는 유지, payload 필드만 확장)
 - **payment-service** : 결제 도메인을 팀 아이디어에 맞게 확장
 - **recommend-service** : 추천 규칙을 팀 도메인에 맞게 확장
- eureka-server / auth-server / api-gateway 는 인프라이므로 학생·실습교수 모두 수정 대상 아님 (단, Eureka 는 서비스 등록 추가 개념을 이해할 수 있게, 소스 이해 제공)
- 인프라 2 종은 tar 파일로 직접 배포 (상세 절차는 별첨 '인프라 이미지 배포 가이드' 참고)
 - Auth Server 와 API Gateway 인프라 2 종은 이미지로 배포하고 소스 기반 실습에서는 제외, SKALA 훈련들은 Java/ SpringBoot 에서 사용자 인증·암호화·JWT 까지는 배우지만 OAuth2 (Authorization Code, Client Credentials) 확장 및 서비스 간 인증, JWK (JWT 서명 검증 공개키) 개념 등으로, 여기까지 실습 범위에 넣으면 난이도가 갑자기 급상승하기 때문에 수위를 조절합니다. (*이제 Java/Spring Boot 를 막 배운 상태로 가정)

상세 일정

시간	Day 1	Day 2
09:00-11:50	Agile 방법론 이론 (Scrum/Kanban, Sprint-Backlog-Standup-Review-Retro, User Story, DoD)	Day1 회고 리마인드 + Sprint2 Planning
	시나리오/아키텍처 설명 + 완성 코드 템플릿 데모 (docker compose up, Eureka 대시보드, 로그인→강의조회→수강신청→결제→추천 전체 흐름)	Sprint1 실행 (오전), Sprint2 착수
12:00-13:20	점심	점심
13:20-14:50	완성 코드 템플릿 설명 (docker compose up, Eureka 대시보드, 로그인→강의조회→수강신청→결제→추천 전체 흐름)	Sprint2 실행 (오후)
	팀 편성 + 아이디어 기획 킥오프	
16:00-18:00	Sprint1 Planning (팀별 시나리오 확정, 담당 역할/디렉토리 배분) Sprint1 실행 Sprint1 Review (데모) + Retro	통합 테스트 및 버그픽스 (버퍼) 최종 데모(팀별 발표) + 전체 회고

Sprint1 / Sprint2 기준

Sprint1은 팀이 기획한 새 도메인을 course-service-enrollment-service 등 서비스 2~3개 안에서만 좁게 적용해, 팀 아이디어에 맞게 로그인부터 신청까지 한 시나리오가 처음부터 끝까지 동작하게 만드는 것이 목표입니다. payment-kafka-recommend는 원본 그대로 두고 통과시킵니다.

Sprint2는 Sprint1에서 손대지 않은 나머지 서비스(결제, Kafka 이벤트, 추천)까지 팀 아이디어에 맞게 확장해 전체 플로우를 완성하는 것이 목표입니다.

인프라 배포 방식 (요약)

담당 전임 교수가 auth-server-api-gateway 2개 이미지를 빌드해 infra-images.tar로 저장하고, 슬랙을 통해 실습 자료와 함께 각 반에 게시합니다. 배포 패키지 구성 : infra-images.tar + 수정된 docker-compose.yml + 학생이 실제로 수정하는 서비스 소스(course-service, enrollment-service, payment-

service, recommend-service, vue-frontend, init-db). 단, 예고 드렸듯이 auth-server 및 api-gateway 디렉토리는 수업 난이도가 높아져서 이번에는 제외합니다.

이해도를 높이기 위한, Day1 설명 보완 사항

학생들은 Java-Spring Boot 기초와 사용자 인증·암호화·JWT까지는 학습했지만, OAuth2·Eureka·Kafka·Gateway·FastAPI·Vue는 이번에 처음 접합니다. 이에 따라 Day1 이론 시간에 "지금까지 배운 JWT 인증에서, 서비스가 여러 개로 쪼개지면 무엇이 새로 필요해지는가?" (토큰 발급자와 검증자의 분리, JWT, 헤더 전파)를 짧게 연결하는 개념 설명을 포함합니다.

이를 통해 학생들이 코드를 직접 수정하지 않아도 auth-server/gateway 코드 리뷰를 이해할 수 있도록 준비합니다.

각 반별 체크리스트

- ☐ 각 반에서 개인별 infra-images.tar + 수정된 docker-compose.yml + 학생용 소스 (course/enrollment/payment/recommend/vue-frontend/init-db) 전달 확인
- ☐ 각 반에서 개인별 docker load + docker compose up -d 동작 테스트 점검
- ☐ eureka-server / auth-server / api-gateway 는 학생이 수정하지 않는다는 점 실습 시작 시 공지
- ☐ Sprint1 : 팀별 기획 시나리오 1 개 확정 및 담당 서비스 역할/디렉토리 배분 확인
- ☐ Sprint1 종료 시 팀별 짧은 데모 + 미니 회고 진행
- ☐ Sprint2 : payment/kafka/recommend 확장 범위 확정, 필요 시 신규 서비스+게이트웨이 라우팅 계획 포함
- ☐ Day2 최종 데모 및 전체 회고 진행

실습 배포 패키지 (전달 항목)

항목	내용
이미지 파일	infra-images.tar (auth-server, api-gateway)
docker-compose.yml	각 서비스 도메인을 컨테이너 이미지로 빌드 배포하는 구성 정보
소스 디렉토리	course-service, enrollment-service, payment-service, recommend-service, vue-frontend, init-db (학생이 실제로 수정하는 것만)
제외 항목	auth-server/, api-gateway/ 디렉토리는 수업 난이도에 맞지 않아 이번에는 제외

실습 시, 개인 PC 공통 실행 절차

```
docker load -i infra-images.tar

# msa-lecture/auth-server:1.0 등 태그 확인

docker images

# 이미지 빌드 및 컨테이너 생성
docker compose build --no-cache
docker compose up -d

## 또는 한줄로
docker compose build --no-cache && docker compose up -d
```

이후 course-service 등 학생이 수정한 서비스만 `docker compose up -d --build <서비스명>`으로 재 빌드하면 됩니다.

이미지 버전 업데이트가 필요할 때 (버그 수정 등)

```
# 1) 재빌드 및 태그 상황
docker build -t msa-lecture/auth-server:1.1 ./auth-server

# 2) 변경된 이미지만 다시 tar로 저장하거나, 전체를 다시 묶어서 저장
docker save -o infra-images-v1.1.tar \
  msa-lecture/auth-server:1.1 \
  msa-lecture/api-gateway:1.0

# 3) docker-compose.yml의 auth-server 태그를 1.1로 수정 후 재배포

# 4) 개인 PC
docker load -i infra-images-v1.1.tar
docker compose up -d
```